AI6101 Reinforcement Learning Assignment

Tabular Q-learning for the CliffBoxPushing Grid World

Name: YourName Matriculation number: YourMatriculationNumber

1 Problem and overview

The agent must push a box from $(1, 1)$ to the goal $(4, 13)$ on the fixed 6×14 map without the agent or the box entering a cliff cell. The state is $s = (a_r, a_c, b_r, b_c)$ (agent and box positions), the actions are up/down/left/right, and the episode ends at the cliff, at the goal, or after 100 steps. The reward of each step is $r_t = -1 - \|a_t - b_t\|_1 - \|b_t - g\|_1$, plus $+1000$ when the box reaches the goal and -1000 for a cliff. Two features of `step()` make the task hard: the box can never be on row 0, row 5 or column 0, and the only feasible corridor passes right next to several cliffs. The box has to be dropped through the single gap in column 3, lifted over the column-6 cliff, slid along row 1 and pushed down column 13.

I implemented **tabular Q-learning** (`RLAgent`), the V-table/policy visualisation, and the action-noise robustness study, plus both bonus tasks (exploration strategies; sparse reward). The game itself (map, reward, `step()`) is never modified. Section 7 lists every change made to the agent and the training/evaluation loops. All numbers below come from the submitted notebook.

2 Method: Q-learning

Q-learning [1] is off-policy TD control. After each transition (s, a, r, s'),

$$Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)], \quad y = \begin{cases} r & \text{if } s' \text{ is a cliff or goal state} \\ r + \gamma \max_{a'} Q(s', a') & \text{otherwise.} \end{cases}$$

The behaviour policy is ε-greedy with random tie-breaking. Hyper-parameters: $\alpha = 0.1, \gamma = 0.99, Q_0 = 0$, 15,000 episodes, and ε decaying linearly from 1.0 to 0.01 over the first half of training. Three implementation details matter:

- **Time-outs are truncations, not terminal states.** Time is not part of the state, so treating the 100-step cut-off as terminal would wrongly teach the agent that some ordinary states have zero future value. The notebook therefore bootstraps at a time-out and stops bootstrapping only at the cliff or the goal (partial-episode bootstrapping [2]).
- **$Q_0 = 0$ is optimistic.** Every non-terminal reward is negative, so untried actions look better than tried ones. This drives systematic exploration on top of ε-greedy (Section 5 isolates this effect).
- **Action noise is part of the environment.** For the robustness study, the agent picks a, the loop may replace it with another action, and the update is applied to the **chosen** a. $Q(s, a)$ therefore estimates the value of choosing a under noise, which is what a noise-aware policy needs.

Sanity check. The reachable state space is small (2,739 non-terminal states), so the notebook also solves the 100-step problem exactly by finite-horizon dynamic programming. This is used **only as a reference**, never for training. The optimal return is **577**. With action noise p, the same DP gives the best achievable expected return and success probability: OPT_TABLE.

3 Results on the original task

3.1 Learning progress

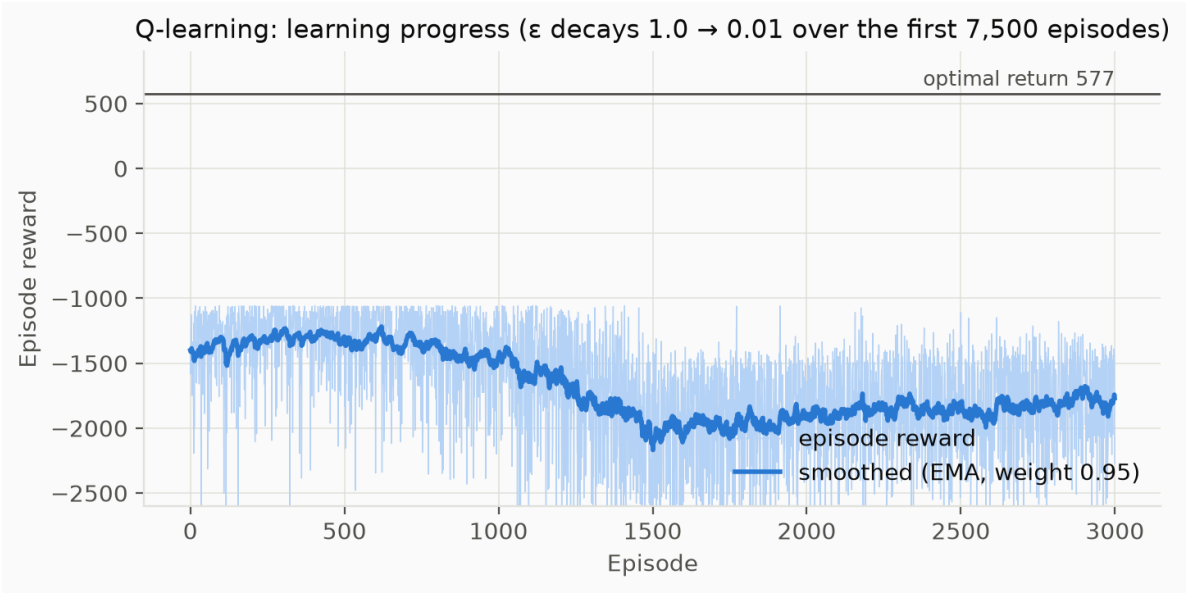


Figure 1: Episode reward vs. episode for the Q-learning agent (light: raw, dark: EMA-smoothed with weight 0.95).

LEARNING_TEXT

3.2 Learned policy and V-table

The template's `visualize()` was completed with $V(s) = \max_a Q(s,a)$ and $\pi(s) = \arg\max_a Q(s,a)$. It prints the 20 largest values, which all belong to states with the box one push from the goal. Because the state is 4-D, I show the policy in three complementary ways.

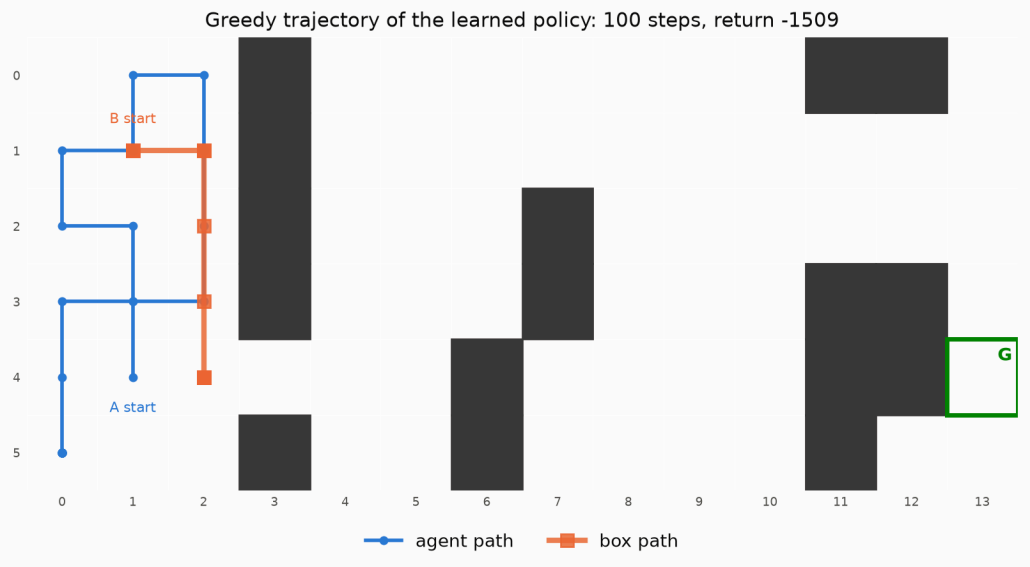


Figure 2: Greedy trajectory of the learned policy (no exploration, no noise): TRAJ_CAPTION

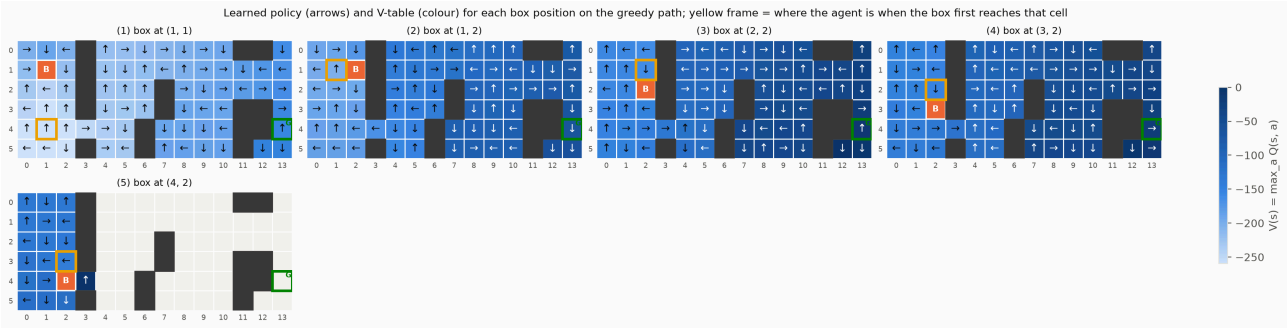


Figure 3: Policy maps. One panel per box position visited by the greedy trajectory. Each agent cell is coloured by $V(s)$, the arrow is the greedy action $\pi(s)$, grey cells were never visited, and the yellow frame is where the agent stands when the box first enters that cell.

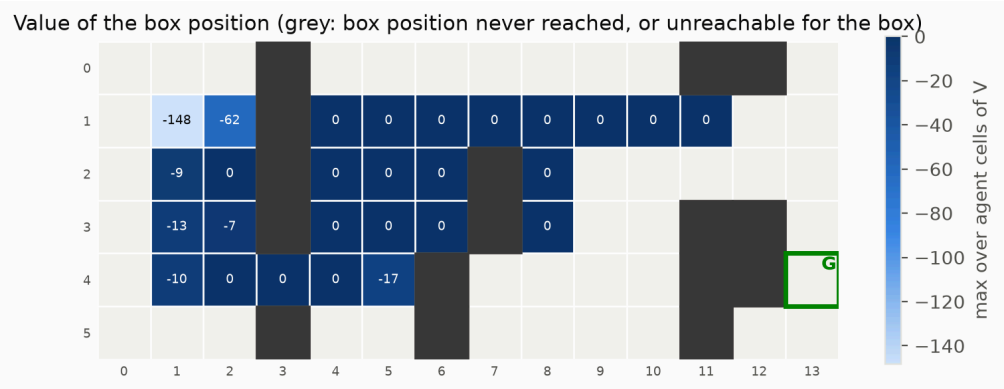


Figure 4: Value of each box position: $\max_{\text{agent cell}} V(\text{agent}, \text{box})$. Grey: never reached.

The policy maps show the structure of the solution. For every box position, the arrows lead the agent around the box to the correct pushing side without stepping into a cliff, and the yellow frame is always the cell behind the box in the direction of the next push. The box-value map rises monotonically along the corridor towards the goal. Greedy evaluation over 1,000 episodes gives a success rate of **MAIN_EVAL_SR**.

4 Action-noise robustness study

Protocol. With probability p, the selected action is replaced by one of the other three actions uniformly at random. This happens in the training/evaluation loop; `step()` is unchanged. For each $p_{\text{train}} \in \{0, 0.1, 0.2\}$ I trained **three seeds** with exactly the main agent's configuration, under two budgets: the template's 15,000 episodes and a long budget of 300,000 episodes. Every policy was evaluated greedily ($\varepsilon = 0$, only the specified noise) for **1,000 episodes** under each $p_{\text{test}} \in \{0, 0.1, 0.2\}$. All tables report the mean over seeds.



Figure 5: Training curves under action noise (mean over 3 seeds; band: min–max over seeds). Top: episode reward. Bottom: training success rate.

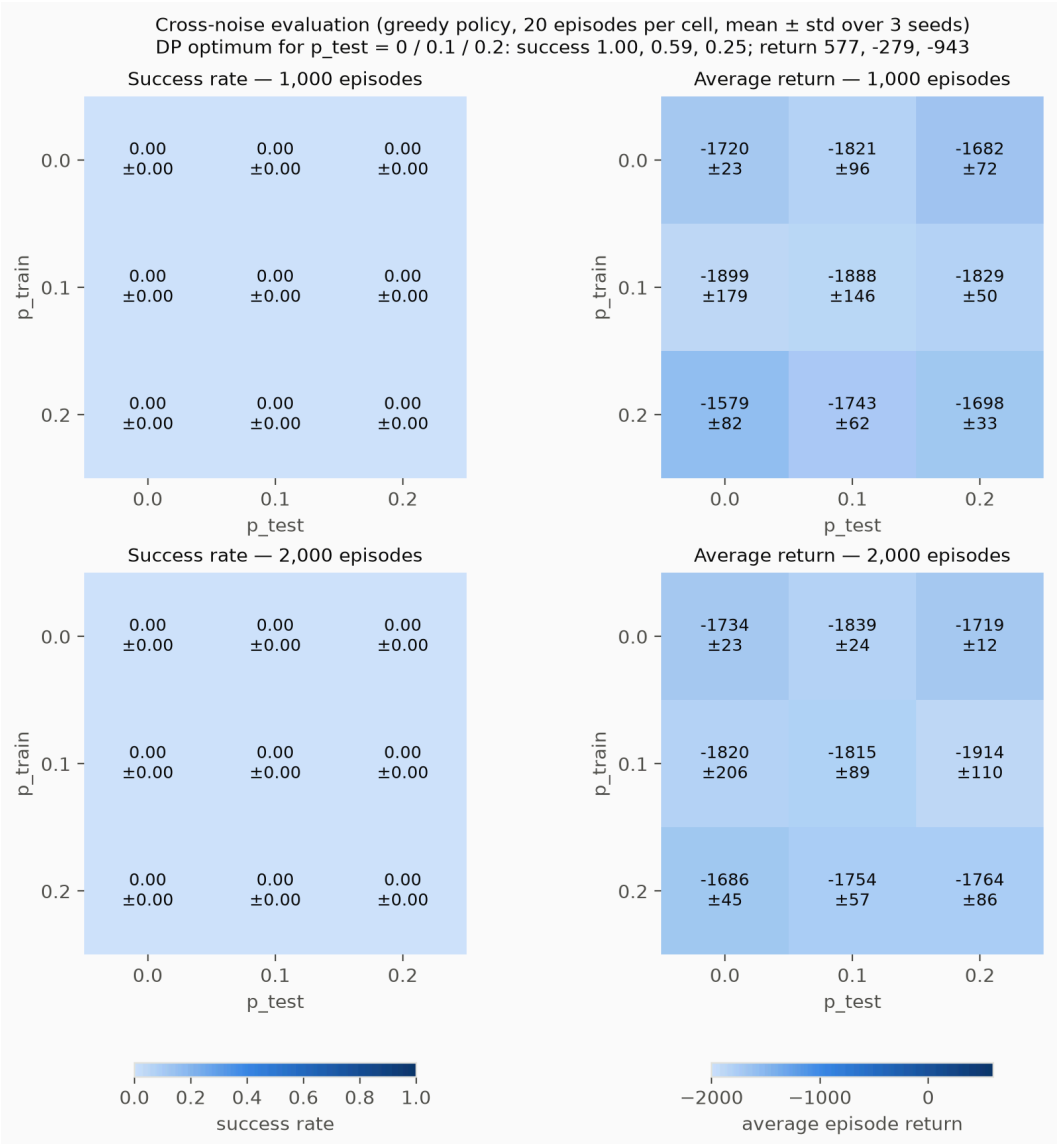


Figure 6: Cross-noise evaluation matrices (rows p_{train}, columns p_{test}; mean ± std over 3 seeds, 1,000 episodes per cell).

NOISE_TEXT

5 Bonus A: exploration strategies

All agents share the Q-learning update, α, γ, the 15,000-episode budget and three seeds (no action noise). Only the behaviour policy changes: decaying ε-greedy (main agent), constant $\varepsilon = 0.1$ (the template default), UCB1 [3] ($\arg\max_a Q(s,a) + c\sqrt{\ln N(s)/N(s,a)}$, $c = 10$, untried actions first), and **no exploration** (always greedy) with the default $Q_0 = 0$ or with a pessimistic $Q_0 = -3000$.

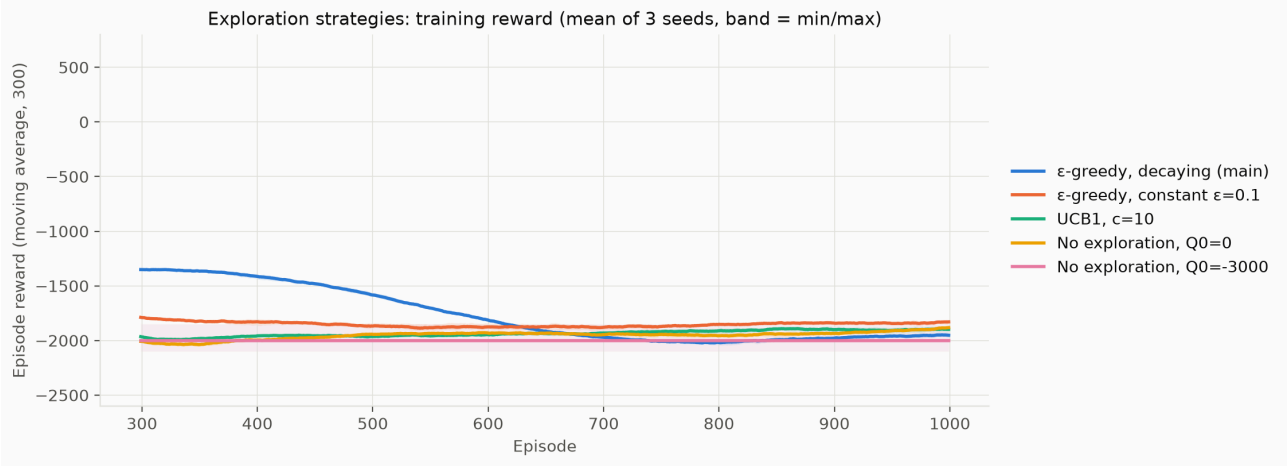


Figure 7: Training reward of the exploration strategies (moving average over 300 episodes, mean of 3 seeds, band: min–max).

EXPLORATION_TABLE

EXPLORATION_TEXT

6 Bonus B: sparse reward

Here the learning signal is **only** $+1000$ when the box reaches the goal: no step cost, no distance shaping and no cliff penalty, although falling still ends the episode. Until the first success every update is $0 \to 0$, so an ε-greedy agent (or a greedy agent with $Q_0 = 0$) performs a random walk, and a random walk essentially never moves the box 15+ cells through the cliff corridor.

Idea: deep exploration through optimistic initial values. Initialise $Q_0 > 0$ and act greedily. With zero rewards, the update $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\gamma \max_{a'} Q(s',a')$ wears down the value of state-actions that have been tried, while untried ones keep their optimistic value. Through the max in the bootstrapped target, this optimism propagates **backwards**, so the agent deliberately travels multi-step routes to the frontier of unexplored states instead of dithering locally, in the spirit of R-max [4]. Falling off the cliff ends the episode and forfeits all future optimism, so cliff avoidance is learned without any penalty. Once the goal is found, its real $+1000$ takes over.

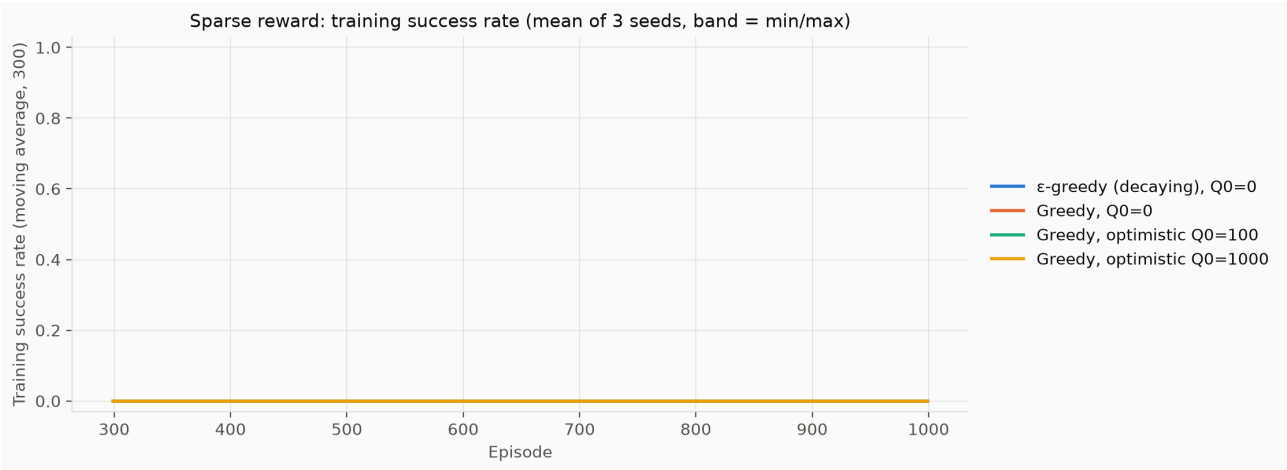


Figure 8: Sparse reward: training success rate (moving average over 300 episodes, mean of 3 seeds, band: min–max).

SPARSE_TABLE

SPARSE_TEXT

7 Summary of modifications

None of the following changes the game (map, rewards, `step()`). Their effect is quantified in the sections cited.

Change	**Reason / evidence**
Linearly decaying ε ($1.0 \longrightarrow 0.01$)	Constant $\varepsilon = 0.1$ keeps losing episodes to random cliff steps until the end of training (Section 5).
Bootstrapping at time-outs	Time is not in the state, so the time limit is a truncation, not a terminal state [2].
Action noise applied after the choice; update on the chosen action	Learns the value of choosing a in the noisy MDP, so noise-aware policies are possible (Section 4).
Long budget (300k episodes) for noisy training	With 15k episodes the noisy agents never find the goal (Section 4).
`sparse_reward` flag and `q_init` parameter	Needed for the Bonus B ablation (Section 6).
NumPy compatibility line `np.chararray = np.char.chararray`	NumPy ≥ 2.5 removed the alias used by the provided renderer. The environment code itself is untouched.

8 Conclusion

CONCLUSION_TEXT

Bibliography

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992.

[2] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time Limits in Reinforcement Learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[3] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, pp. 235–256, 2002.

[4] R. I. Brafman and M. Tennenholtz, "R-max: A General Polynomial Time Algorithm for Near-Optimal Reinforcement Learning," *Journal of Machine Learning Research*, vol. 3, pp. 213–231, 2002.